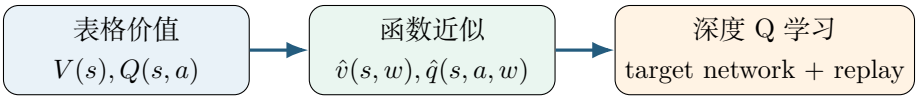
第 8 章：Value Function Methods

初学者快速认识与理解指南



核心结论

本章只需先抓住一个变化：L7 更新一个表项，L8 更新一组共享参数。

$$w \leftarrow w + \alpha(\text{target} - \text{当前估计})\nabla_w(\text{当前估计}).$$

target 仍区分 TD、Sarsa 与 Q-learning；函数梯度则把同一条经验推广到其他状态。

基于 Book Chapter 8（原书第 151–189 页）与 L8 连续更新 slides 整理

覆盖：线性 TD、投影 Bellman 方程、Sarsa/Q-learning 函数近似与 DQN

目录

8 L8：Value Function Methods——从表格走向函数 2
8.1 为什么表格不够：存储、查询、更新与泛化 . . . 2
8.2 三类常见逼近器：线性、特征展开与神经网络 . . . 2
8.3 先把目标函数说清楚：状态分布决定“哪里更重要” . . . 3
8.4 从监督式 SGD 到半梯度 TD . . . 3
8.5 TD-Linear：把梯度换成特征 . . . 4
8.6 TD-Linear 真正在解什么：投影 Bellman 方程 . . . 4
8.7 LSTD：直接估计线性方程 . . . 5
8.8 带函数近似的 Sarsa：评价实际执行的下一动作 . . . 5
8.9 带函数近似的 Q-learning：target 仍然取最大值 . . . 5
8.10 为什么普通深度 Q-learning 需要稳定化 . . . 6
8.11 DQN 技术一：主网络与目标网络 . . . 6
8.12 DQN 技术二：经验回放 . . . 6
8.13 DQN 的网络输出、训练循环与损失解释 . . . 7
8.14 L7 到 L8：只看 target 与更新对象就不会迷路 . . . 7
8.15 初学者最容易产生的八个误解 . . . 8
8.16 自测题与参考答案 . . . 8
8.17 一页总结 . . . 9

内容核对说明 10

8 L8：Value Function Methods——从表格走向函数

前七讲默认用表格保存价值：每个状态对应一个 $V(s)$，每个状态–动作对对应一个 $Q(s,a)$。这种表示清楚、定理也容易分析，但当状态连续或数量极大时，表格既存不下，也无法给从未访问过的状态估值。L8 的核心变化是用带参数的函数代替表格：

$$v_\pi(s) \approx \hat{v}(s,w), \qquad q_\pi(s,a) \approx \hat{q}(s,a,w),$$

其中 $w \in \mathbb{R}^m$ 是需要从样本中学习的参数。

L8 的一句话主线

把 L7 的“只修改一个表项”改成“沿函数梯度修改参数”：

$$w_{t+1} = w_t + \alpha_t \underbrace{(\text{target} - \text{当前估计})}_{\text{TD error}} \underbrace{\nabla_w \hat{v} \text{ 或} \nabla_w \hat{q}}_{\text{把误差传给参数}}.$$

target 决定算法在逼近哪个 Bellman 方程；梯度决定一次误差怎样影响整个函数。

8.1 为什么表格不够：存储、查询、更新与泛化

比较维度	表格表示	函数表示
存储	保存 $\lvert\mathcal{S}\rvert$ 或 $\lvert\mathcal{S}\rvert\lvert\mathcal{A}\rvert$ 个数	只保存 m 个参数，通常 m 远小于表项数
查询	按索引读取一个表项	做一次前向计算 $\hat{v}(s,w)$ 或 $\hat{q}(s,a,w)$
更新	通常只改变刚访问的表项	修改 w，因而可能同时改变许多状态的估值
未见状态	没有样本的表项无法自行推断	相似特征可共享参数，产生**泛化**
代价	精确表示有限表格，但规模受限	节省存储并可泛化，但产生**近似误差**

直观理解

函数近似相当于把许多状态“绑”在同一组参数上。一次更新不仅修正 S_t，也会改变与 S_t 使用相似特征的状态。泛化正是由此产生；相互干扰也由此产生。

8.2 三类常见逼近器：线性、特征展开与神经网络

最重要的线性形式是

$$\hat{v}(s,w) = \phi(s)^\top w, \qquad \hat{q}(s,a,w) = \phi(s,a)^\top w,$$

其中 ϕ 是预先设计的特征。这里的“线性”是指对参数 w 线性；$\phi(s)$ 本身完全可以包含 s^2、Fourier 基、tile coding 等非线性变换。神经网络则直接用非线性函数表示 $\hat{v}$ 或 $\hat{q}$，表达能力更强，但理论分析与训练稳定性更复杂。

表格其实是线性函数近似的特例

若有 n 个状态，令 $\phi(s_i) = e_i$ 为第 i 个单位向量，则

$$\hat{v}(s_i, w) = e_i^\top w = w_i.$$

参数向量 w 就是原来的价值表。因此 L8 不是抛弃表格方法，而是把它包含在更一般的表示中。

如果所有真实状态价值已经知道，把状态逐行组成设计矩阵 Φ，普通最小二乘解为

$$w_{\mathrm{LS}} = (\Phi^\top \Phi)^{-1} \Phi^\top v_\pi,$$

前提是 $\Phi^\top \Phi$ 可逆。RL 的困难恰恰是 v_π 通常未知，只能从交互样本构造学习目标。

8.3 先把目标函数说清楚：状态分布决定“哪里更重要”

固定策略 π 时，一个自然目标是加权价值均方误差

$$J_E(w) = \mathbb{E}_{S\sim d}\big[(v_\pi(S) - \hat{v}(S, w))^2\big] = \|v_\pi - \Phi w\|_D^2,$$

其中 $D = \mathrm{diag}(d)$，且 $\|x\|_D^2 = x^\top D x$。分布 d 不是装饰：它决定哪些状态的误差在总目标中权重更大。

对课程重点的 on-policy continuing setting，常用策略 π 的平稳分布 d_π：

$$d_\pi^\top = d_\pi^\top P_\pi, \qquad \sum_s d_\pi(s) = 1.$$

直观上，长期运行策略后经常访问的状态会得到较大权重。该解释依赖马尔可夫链具有合适的不可约性、遍历性等条件；不能在任意链上无条件假定唯一平稳分布存在。

“最小误差”必须先说明按什么分布

同一逼近器在均匀分布下的最优参数，未必是在 d_π 下的最优参数。因此比较两个函数逼近器时，必须同时说明特征、数据分布和误差度量。

8.4 从监督式 SGD 到半梯度 TD

若能观察到标签 $v_\pi(S_t)$，对 J_E 做随机梯度下降可写成

$$w_{t+1} = w_t + \alpha_t [v_\pi(S_t) - \hat{v}(S_t, w_t)] \nabla_w \hat{v}(S_t, w_t).$$

但真实价值未知。教材依次给出两种可采样替代：

- 用完整回报 G_t 替代，得到带函数近似的 Monte Carlo 更新；
- 用一步 TD target $R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t)$ 替代，得到

$$\boxed{w_{t+1} = w_t + \alpha_t \delta_t \nabla_w \hat{v}(S_t, w_t)}, \qquad \delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t).$$

为什么叫 semi-gradient（半梯度）

TD target 中的 $\hat{v}(S_{t+1}, w_t)$ 也依赖 w_t，但更新只对当前估计 $\hat{v}(S_t, w_t)$ 求梯度，把 target 暂时当成常数。因此它不是对“单样本 TD error 平方”做完整梯度下降，也不能据此宣称它直接最小化 J_E。教材随后用投影 Bellman 方程给出严格解释。

一次线性 TD 更新

设 $\phi(S_t) = (1,2)^\top$，$\phi(S_{t+1}) = (1,1)^\top$，$w_t = (0.5,1)^\top$，$R_{t+1} = 1$，$\gamma = 0.9$，$\alpha = 0.1$。则

$$\hat{v}(S_t, w_t) = 2.5, \qquad \hat{v}(S_{t+1}, w_t) = 1.5,$$

$$\delta_t = 1 + 0.9 \times 1.5 - 2.5 = -0.15, \qquad w_{t+1} = w_t + 0.1(-0.15)(1,2)^\top = (0.485, 0.970)^\top.$$

一次样本同时改变两个参数，之后所有使用这两个参数的状态估值都可能改变。

8.5 TD-Linear：把梯度换成特征

在线性表示下 $\nabla_w \hat{v}(s,w) = \phi(s)$，于是

$$\boxed{w_{t+1} = w_t + \alpha_t \big[R_{t+1} + \gamma \phi(S_{t+1})^\top w_t - \phi(S_t)^\top w_t\big] \phi(S_t)}.$$

这就是课程中的 TD-Linear。它保留 L7 的“一步采样加自举”，只是把“改一个表项”换成“沿当前状态的特征方向改参数”。若 S_{t+1} 为终止状态，约定其 bootstrap 价值为零。

8.6 TD-Linear 真正在解什么：投影 Bellman 方程

令

$$T_\pi v = r_\pi + \gamma P_\pi v, \qquad \Pi_D = \Phi(\Phi^\top D \Phi)^{-1} \Phi^\top D,$$

其中 Π_D 把任意价值向量按 D-加权距离投影到特征子空间 $\{\Phi w : w \in \mathbb{R}^m\}$。TD-Linear 的极限满足

$$\boxed{\Phi w^* = \Pi_D T_\pi(\Phi w^*)},$$

即“做一次 Bellman 更新，再投影回函数类”后的固定点。

教材比较了三个容易混淆的目标：

$$\begin{aligned} J_E(w) &= \|\Phi w - v_\pi\|_D^2, && \text{价值近似误差,} \\ J_{BE}(w) &= \|\Phi w - T_\pi(\Phi w)\|_D^2, && \text{Bellman error,} \\ J_{PBE}(w) &= \|\Phi w - \Pi_D T_\pi(\Phi w)\|_D^2, && \text{projected Bellman error.} \end{aligned}$$

TD-Linear 对应第三种投影固定点，而不是一般意义下直接最小化第一种目标。

直观理解

真实 Bellman 更新 $T_\pi(\Phi w)$ 往往已经离开特征子空间，现有函数类根本表示不了它。投影 Π_D 的作用是把它拉回可表示集合，所以 TD 最后解的是“Bellman 更新 + 投影”的固定点。

在 on-policy 平稳采样下，线性 TD 的期望更新可整理为

$$w_{t+1} = w_t + \alpha_t(b - A w_t),$$

$$A = \Phi^\top D(I - \gamma P_\pi)\Phi, \qquad b = \Phi^\top D r_\pi.$$

在教材给出的适当条件下，w_t 收敛到

$$w^* = A^{-1}b.$$

典型前提包括：线性特征具有足够秩、按目标策略的平稳分布采样、奖励有界，且步长满足随机逼近所需条件。还可得到近似误差界

$$\|\Phi w^* - v_\pi\|_D \le \frac{1}{1-\gamma} \min_w \|\Phi w - v_\pi\|_D.$$

不要把线性 on-policy 定理无限外推

上述结论针对固定策略、线性函数近似和相匹配的 on-policy 分布。它不自动证明非线性神经网络 TD、off-policy 评价或函数近似 Q-learning 一定收敛。对超出定理范围的算法，应区分“实践中常用”与“已有该定理保证”。

8.7 LSTD：直接估计线性方程

既然极限满足 $Aw = b$，Least-Squares TD（LSTD）的思路就是用样本累计

$$\widehat{A} = \sum_t \phi_t(\phi_t - \gamma\phi_{t+1})^\top, \qquad \widehat{b} = \sum_t R_{t+1}\phi_t,$$

再解 $\widehat{A}w = \widehat{b}$。它能充分复用样本，常比逐步小步长更新更快；代价是存储矩阵和求解线性系统的计算量更大，且这里讨论的是线性、固定策略状态价值评价。

8.8 带函数近似的 Sarsa：评价实际执行的下一动作

把 L7 的 Sarsa 表格更新换成参数更新：

$$\boxed{w_{t+1} = w_t + \alpha_t \delta_t^{\mathrm{Sar}} \nabla_w \hat{q}(S_t, A_t, w_t)},$$

$$\delta_t^{\mathrm{Sar}} = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, w_t) - \hat{q}(S_t, A_t, w_t).$$

若 $\hat{q}(s,a,w) = \phi(s,a)^\top w$，梯度就是 $\phi(S_t, A_t)$。控制时仍由当前 $\hat{q}$ 构造 ε-greedy 策略，并由该策略实际生成 A_{t+1}，因此标准 Sarsa 是 on-policy。

Sarsa 函数近似的最小记忆单元

$$\text{target} = R + \gamma \hat{q}(S', A', w), \quad \text{error} = \text{target} - \hat{q}(S, A, w), \quad w \leftarrow w + \alpha\, \text{error}\, \nabla_w \hat{q}(S, A, w).$$

结构与 L7 完全相同；新增的只是最后一个梯度因子。

8.9 带函数近似的 Q-learning：target 仍然取最大值

Q-learning 的参数更新为

$$\boxed{w_{t+1} = w_t + \alpha_t \delta_t^{Q} \nabla_w \hat{q}(S_t, A_t, w_t)},$$

$$\delta_t^{Q} = R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, w_t) - \hat{q}(S_t, A_t, w_t).$$

行为动作 A_t 可以由探索策略产生，但 target 评价的是下一状态的贪心动作，因此按标准 backup 分类它是 off-policy。

方法	target 的下一步部分	参数更新方向
Sarsa	$\hat{q}(S', A', w)$，A' 为实际采样动作	$\nabla_w \hat{q}(S, A, w)$
Q-learning	$\max_a \hat{q}(S', a, w)$	$\nabla_w \hat{q}(S, A, w)$

与 L7 的术语保持一致

课程材料会按“边采样边更新”或“先由行为策略收集经验再更新”描述不同实现版本；但只要 backup 使用 $\max_a \hat{q}(S', a, w)$，标准 RL 分类仍把它看作 off-policy target。online/offline 描述数据何时收集，on/off-policy 描述行为策略与学习目标是否一致，二者不是同一维度。

8.10 为什么普通深度 Q-learning 需要稳定化

若直接令神经网络参数 w 同时出现在当前预测和 target 中，可写出形式目标

$$\mathbb{E}\Big[\big(R + \gamma \max_a \hat{q}(S', a, w) - \hat{q}(S, A, w)\big)^2\Big].$$

此时网络每更新一次，预测和训练标签都会一起移动，样本又按轨迹连续出现，普通监督学习的“固定标签、近似独立样本”直觉不再成立。教材中的 DQN 用两项关键技术缓解这两个问题：

1. **target network**：暂时冻结生成 target 的参数；
2. **experience replay**：从经验池均匀随机抽取 mini-batch。

8.11 DQN 技术一：主网络与目标网络

主网络参数记为 w，目标网络参数记为 w_T。对一条经验 (s, a, r, s')，课程版本的 target 为

$$y_T = r + \gamma \max_{a'} \hat{q}(s', a', w_T),$$

然后只更新主网络，使

$$L(w) = \big(y_T - \hat{q}(s, a, w)\big)^2$$

减小。每隔 C 次主网络更新，再复制 $w_T \leftarrow w$。若 s' 为终止状态，应令 bootstrap 项为零；代码中常写成

$$y_T = r + \gamma(1 - \texttt{done}) \max_{a'} \hat{q}(s', a', w_T).$$

直观理解

冻结 w_T 的一段时间内，y_T 对主网络参数 w 是固定标签，梯度只需穿过 $\hat{q}(s, a, w)$。定期复制让标签跟随学习进度，但不会在每个微小更新中立即追着主网络移动。

8.12 DQN 技术二：经验回放

把行为策略生成的经验存入 replay buffer

$$\mathcal{B} = \{(s, a, r, s')\},$$

训练时不按收集顺序使用，而是从 $\mathcal{B}$ 中均匀随机抽取 mini-batch。这样做有三层作用：

- 打散相邻轨迹样本的强时间相关性；
- 同一经验可被多次利用，提高样本利用率；
- 让 mini-batch 梯度更接近课程目标中对经验分布的平均。

经验通常由带探索的行为策略产生，target 却取贪心最大值，所以 DQN 是 off-policy 方法。

经验回放不是“数据随便放进去就行”

经验池仍需覆盖学习任务所需的状态–动作区域。若行为策略从未访问关键动作，随机抽样不能凭空创造信息。课程算法强调均匀回放；其他采样方案需要相应修正与额外分析，不应与本章基础版本混写。

8.13 DQN 的网络输出、训练循环与损失解释

离散动作下常见两种网络接口：

- 输入 (s, a)，输出一个标量 $\hat{q}(s, a, w)$；
- 只输入 s，一次输出所有离散动作的 Q 值向量，便于计算 $\arg\max_a$。

DQN 基础循环

1. 用带探索的行为策略与环境交互，把 $(S, A, R, S', \texttt{done})$ 放入经验池；
2. 均匀抽取 mini-batch；
3. 用目标网络计算每条样本的 y_T，终止样本不 bootstrap；
4. 用 mini-batch 降低 $(y_T - \hat{q}(S, A, w))^2$，只更新主网络；
5. 每隔 C 次更新复制 $w_T \leftarrow w$。

训练损失很小，不代表价值函数一定正确

损失只检查经验池中出现的输入以及由当前 target network 生成的标签。如果数据覆盖不足、target 本身偏离真实最优价值，网络完全可能把训练损失降得很低，却仍在未覆盖状态上严重出错。因此损失曲线必须结合回报、覆盖范围和独立评估理解。

8.14 L7 到 L8：只看 target 与更新对象就不会迷路

方法	target	实际被更新的对象
表格 TD	$R + \gamma V(S')$	当前表项 $V(S)$
线性 TD	$R + \gamma \phi(S')^\top w$	参数 w，方向为 $\phi(S)$
Sarsa–FA	$R + \gamma \hat{q}(S', A', w)$	参数 w，方向为 $\nabla_w \hat{q}(S, A, w)$

方法	target	实际被更新的对象
Q-learning–FA	$R + \gamma \max_a \hat{q}(S', a, w)$	参数 w，方向为 $\nabla_w \hat{q}(S, A, w)$
DQN	$R + \gamma \max_a \hat{q}(S', a, w_T)$	主网络 w；w_T 周期性复制

核心结论

L8 的算法名称很多，但只有两次结构变化：

1. 表格变成 $\hat{v}/\hat{q}$ 后，更新末尾多出一个对参数的梯度；
2. DQN 再把 target 中的参数从 w 换成暂时冻结的 w_T，并用经验回放组成 mini-batch。

8.15 初学者最容易产生的八个误解

1. **“线性近似只能拟合状态的直线函数。”**
   线性是对 w 而言；非线性特征 $\phi(s)$ 仍可表达复杂形状。
2. **“函数近似只是更省内存，不会改变更新含义。”**
   参数共享带来泛化，也会让一次更新影响其他状态。
3. **“TD target 替代真实价值后，仍是在对 J_E 做普通 SGD。”**
   不准确。TD 是半梯度更新；线性理论对应投影 Bellman 固定点。
4. **“Bellman error 与 projected Bellman error 是同一个量。”**
   不是。后者先把 Bellman 更新投影回可表示的特征空间。
5. **“表格方法与线性函数近似互不相关。”**
   one-hot 特征使线性表示精确退化为表格。
6. **“线性 TD 的收敛定理也保证任意神经网络 Q-learning 收敛。”**
   定理范围不同，不能外推。
7. **“目标网络是第二个独立学习的目标。”**
   基础 DQN 中它不单独优化，而是周期性复制主网络参数。
8. **“DQN loss 接近零就证明学到了 q_*。”**
   loss 只说明对当前数据和当前 target 拟合得好，不证明数据覆盖与 target 正确。

8.16 自测题与参考答案

题目

1. 为什么状态分布 d 会改变函数近似的最优参数?
2. 写出半梯度 TD 的一般更新，并说明为什么叫“半梯度”。
3. 证明 one-hot 特征下 $\hat{v}(s, w) = \phi(s)^\top w$ 等价于价值表。
4. 写出投影 Bellman 固定点，并说明投影为什么必要。

5. 写出 Sarsa–FA 与 Q-learning–FA 的 target，指出唯一关键差异。
6. DQN 中主网络与目标网络各做什么？
7. 为什么经验回放不能弥补行为策略完全没有覆盖的动作？
8. 训练损失很小但平均回报很差，至少给出两种可能原因。

参考答案

1. J_E 对各状态误差按 $d(s)$ 加权；分布变化会改变不同状态之间的折中。
2. $w \leftarrow w + \alpha[R + \gamma\hat{v}(S', w) - \hat{v}(S, w)]\nabla_w \hat{v}(S, w)$；target 也依赖 w，但求导时把它暂当常数。
3. 令状态 s_i 的特征为单位向量 e_i，则 $e_i^\top w = w_i$，恰好读取第 i 个表项。
4. $\Phi w^* = \Pi_D T_\pi(\Phi w^*)$；Bellman 更新一般不在特征子空间内，必须投影回可表示集合。
5. Sarsa 用 $R + \gamma\hat{q}(S', A', w)$；Q-learning 用 $R + \gamma \max_a \hat{q}(S', a, w)$。差异是实际下一动作与最大动作价值。
6. 主网络通过梯度下降拟合 target；目标网络暂时冻结并生成 target，每隔 C 次从主网络复制参数。
7. 回放只能重采已有经验；经验池里没有的动作仍没有奖励与转移信息。
8. 可能是经验覆盖不足、target 偏差、只在训练样本上拟合、探索不足，或终止状态 bootstrap 处理错误等。

8.17 一页总结

核心结论

1. 函数近似用少量参数表示大量状态价值，获得泛化，同时引入近似误差与状态间干扰。
2. 线性表示是对参数线性；one-hot 特征使它包含表格表示。
3. 价值误差必须连同状态分布一起定义；on-policy 分析常使用策略平稳分布 d_π。
4. 半梯度 TD 更新是“TD error 乘当前估计对参数的梯度”。
5. TD-Linear 的严格目标是投影 Bellman 固定点，而非直接最小化普通价值 MSE。
6. 线性 TD 收敛结论有明确的固定策略、on-policy、特征与步长条件，不能无条件外推。
7. Sarsa–FA 使用实际下一动作，Q-learning–FA 使用下一状态最大动作价值。
8. DQN 用目标网络暂时固定标签，用经验回放打散相关性并复用样本。
9. 终止状态不 bootstrap；DQN 的低训练损失也不等于真实价值误差必然低。
10. 识别算法时先圈出 target，再看更新的是表项、线性参数还是神经网络参数。

掌握标准：能够从任意价值函数更新中指出“逼近器、target、数据分布、梯度方向和定理适用范围”，并能解释目标网络与经验回放各自解决的结构性问题。

内容核对说明

本资料的定义、公式、算法结构与理论结论以工作区内的 `Book` 目录为依据，逐项核对以下文件：

- `3 - Chapter 8 Value Function Methods.pdf`（原书第 151–189 页）；
- `Lecture slides/slidesContinuouslyUpdated/ L8-Value function methods.pdf.pdf`；
- `Lecture slides/slidesForMyLectureVideos/ L8_Value function approximation.pdf`；
- `5 - Errata for the Springer version.pdf`。

专业性处理包括：最小二乘解保留维度正确的 $\Phi^\top v$；明确 TD 更新是半梯度，线性 TD 对应 projected Bellman fixed point，而非直接最小化普通价值 MSE；将线性 on-policy 收敛结论限制在教材假设范围内；统一 Algorithms 8.2–8.3 的 ε-greedy 记号（勘误表所列修正已由 GitHub 版教材吸收）；并补全终止状态不 bootstrap 的实现约定。